

Informe fullstack 3

Caso Grupo Cordillera



Integrantes: Oscar Leyton / Benjamin Vasquez / Patricio Olguin
Profesor: Claudio Rojas
Sección: 003D

Índice

1. Introducción
2. Requerimientos del Sistema
 - 2.1. Historias de Usuario
 - 2.2. Requerimientos Funcionales (RF)
 - 2.3. Requerimientos No Funcionales (RNF)
3. Diseño de Arquitectura (Modelo C4)
 - 3.1. Nivel 1: Diagrama de Contexto
 - 3.2. Nivel 2: Diagrama de Contenedores
 - 3.3 Nivel 3: Diagrama de Componentes
4. Justificación de Patrones de Diseño
 - 4.1. Base de datos por microservicio
 - 4.2. Backend For Frontend (BFF)
 - 4.3. Tolerancia a Fallos (Manejo de Errores en BFF)
 - 4.4. Repository Pattern
 - 4.5 Observabilidad y monitoreo
5. Evaluación Ética y Técnica
6. Descripción de componentes
7. Conclusión

Introducción

En este informe se dará a conocer el diseño arquitectónico para la nueva plataforma de monitoreo del Grupo Cordillera. Este presenta el desafío para la gestión y análisis de la información de sus sistemas (puntos de ventas, inventario) nosotros proponemos un sistema basado en microservicios y la implementación de un bff, gracias a esto el enfoque permitirá consolidar la información, asegurar la disponibilidad de los KPI y facilitar la toma de decisiones gerenciales en tiempo real. Disminuyendo los errores y retrasos.

2. Requerimientos del Sistema

2.1. Historias de Usuario

ID	Rol	Deseo	Propósito / Beneficio
01	Alta Gerencia	Visualizar KPIs de ventas y rentabilidad en tiempo real.	Tomar decisiones estratégicas basadas en datos actualizados, no en reportes manuales.
02	Administrador	Centralizar la información de inventarios de múltiples sucursales.	Evitar quiebres de stock y mejorar la logística nacional.
03	Analista de Datos	Acceder a datos normalizados y poder generar reportes.	Reducir el tiempo de preparación de informes y eliminar errores de transcripción.

2.2. Requerimientos Funcionales (RF)

RF-01: Gestión de Identidad y Acceso: El sistema debe permitir el inicio de sesión y la gestión de perfiles de usuario (Administrador, Ventas y usuarios normales) a través del microservicio de **Usuario**.

RF-02: Consolidación y Centralización de datos: El microservicio de Gestión de datos debe automatizar la extracción y centralización de la información proveniente de los sistemas de Inventario y Compra.

RF-03: Análisis de KPI: El microservicio de KPI debe procesar los datos centralizados para calcular indicadores claves, alineados con los objetivos del negocio.

RF-04: Visualización de panel de control: El sistema debe proporcionar una interfaz gráfica que consuma datos del microservicio de Reportes para mostrar los indicadores estratégicos.

RF-05: Generación de Reportes: El sistema debe permitir al Analista de Datos extraer informes históricos y actuales a través del microservicio de Reportes, para la generación de reportes excel o PDF.

RF-06: Gestión de Carrito de Compras: El sistema debe permitir a los usuarios agregar, eliminar y modificar productos en una sesión activa mediante el microservicio de Carrito.

2.3. Requerimientos No Funcionales (RNF)

RNF-01: Seguridad: El sistema deberá implementar mecanismos de autenticación y autorización basados en roles para garantizar que solo usuarios autorizados accedan a la información y funcionalidades correspondientes.

RNF-02: Rendimiento: La consulta de indicadores y generación de reportes deberá realizarse en un tiempo máximo de 4 segundos bajo condiciones normales de operación.

RNF-03: Escalabilidad: La arquitectura basada en microservicios deberá permitir la incorporación de nuevos servicios o funcionalidades sin afectar el funcionamiento de los componentes existentes.

RNF-04: Disponibilidad: La plataforma deberá mantener una disponibilidad mínima del 99% para asegurar el acceso continuo a los servicios de monitoreo y reportería.

RNF-05: Compatibilidad: La aplicación deberá ser accesible desde los principales navegadores web modernos, como Google Chrome, Microsoft Edge y Mozilla Firefox.

RNF-06: Calidad y Mantenibilidad: El sistema deberá aplicar patrones de diseño, buenas prácticas de programación y pruebas unitarias con una cobertura mínima del 60%, facilitando su mantenimiento y evolución futura.

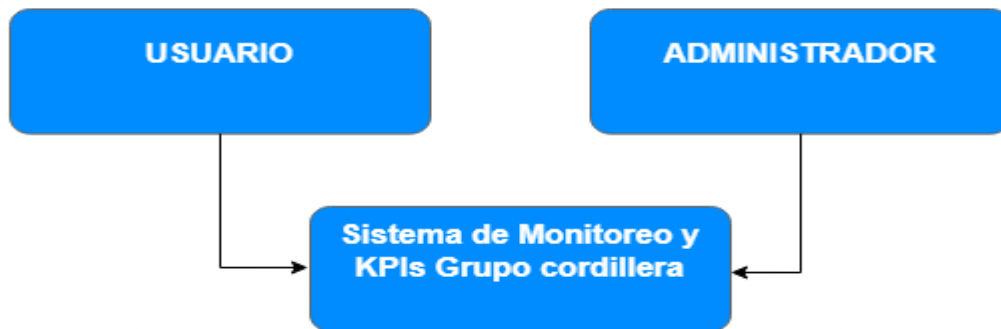
3. Diseño de Arquitectura de Microservicios

Para resolver la dispersión de datos, se propone un ecosistema desacoplado:

3.1. Nivel 1: Diagrama de Contexto

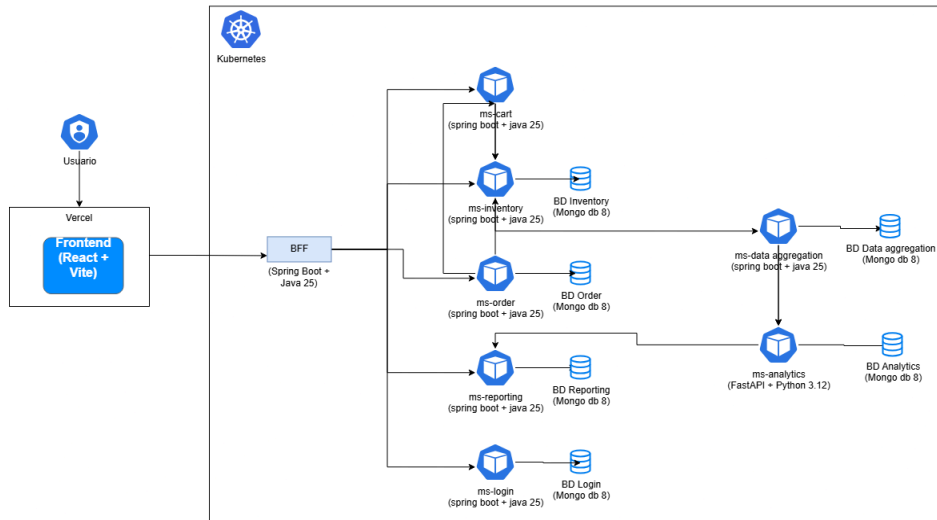
Este nivel muestra cómo el sistema interactúa con los usuarios y sistemas externos.

- **Usuario:** Accede a la plataforma para consultar datos y KPIs.
- **Sistema Grupo Cordillera:** El núcleo de la solución que procesa la información.



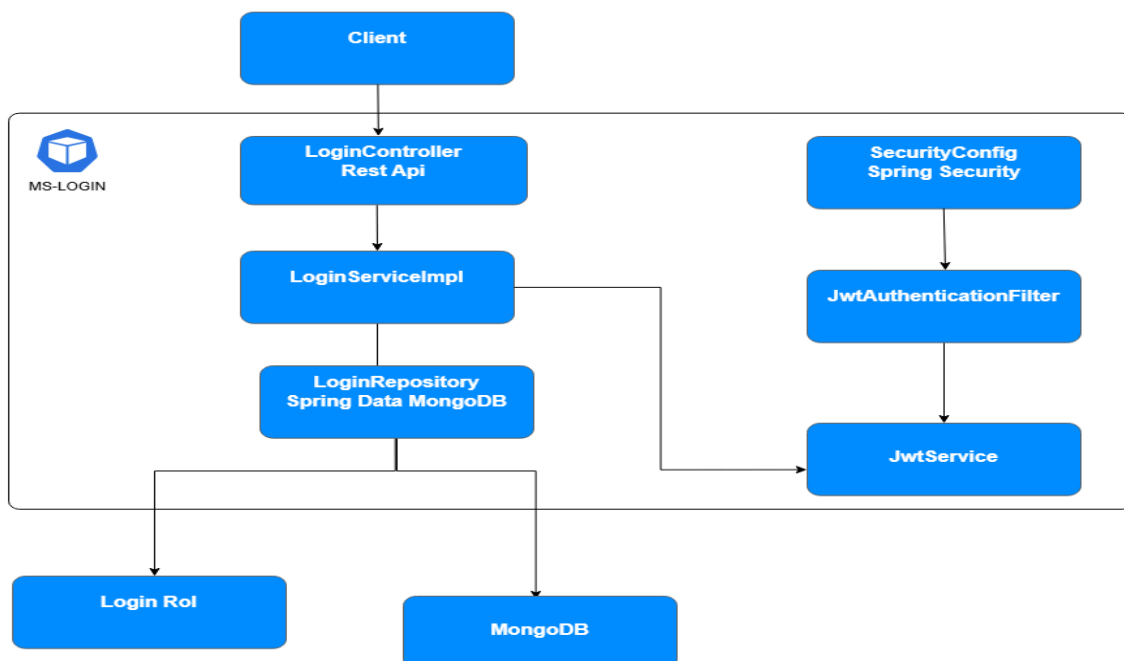
3.2. Nivel 2: Diagrama de Contenedores

Se realiza un desglose del sistema dentro del clúster de Kubernetes, detallando las responsabilidades y tecnologías de cada contenedor.



3.3. Nivel 3: Diagrama de Componentes

El diagrama C3 representa los componentes internos del microservicio en este caso el de ms-login



4. Justificación de Patrones de Diseño

4.1. Base de datos por microservicio

Se aplica el patrón Database per Service, donde cada microservicio posee su propia base de datos en MongoDB. Esto permite un mayor desacoplamiento entre servicios. Además, facilitará la escalabilidad independiente de cada servicio.

4.2. Backend For Frontend (BFF)

Se implementa el patrón **Backend For Frontend (BFF)** mediante el microservicio bff, el cual actúa como punto único de entrada para el frontend. Su función principal es centralizar y coordinar las solicitudes hacia los microservicios de login, inventario y carrito.

Este patrón permite simplificar la comunicación del frontend, evitando el acceso directo a los microservicios internos. Además, mejora la seguridad, el desacoplamiento y la mantenibilidad del sistema, facilitando futuras modificaciones sin afectar la interfaz de usuario.

4.3. Tolerancia a Fallos (Manejo de Errores en BFF)

Se implementa un mecanismo de manejo de errores en el Backend For Frontend (BFF), utilizando interceptores globales mediante `@RestControllerAdvice`. Esto permite capturar excepciones generadas tanto a nivel del BFF como de los microservicios, especialmente mediante el manejo de `FeignException`.

Este enfoque mejora la resiliencia del sistema al entregar respuestas controladas ante fallos de servicios externos, evitando que los errores se propaguen directamente al frontend. Se establece una base de tolerancia a fallos mediante el manejo centralizado de excepciones.

4.4. Repository Pattern

Se implementa el patrón Repository en los distintos microservicios del sistema mediante Spring Data MongoDB. Cada microservicio define su propio repositorio, el cual extiende `MongoRepository`, permitiendo abstraer el acceso a la base de datos y facilitando operaciones CRUD sin necesidad de implementar consultas manuales.

Este patrón se evidencia en componentes como `LoginRepository`, `InventoryRepository` y `OrderRepository`, mejorando la separación de responsabilidades y reduciendo el acoplamiento con la capa de persistencia.

El uso de este patrón contribuye a una arquitectura más limpia, escalable y mantenible dentro del ecosistema de microservicios.

4.5 Observabilidad y monitoreo de errores

Se implementa GlitchTip una herramienta de monitoreo y rastreo de errores, desplegada como servicio independiente dentro del cluster de kubernetes. Cada microservicio y el BFF se configuran como proyectos independientes dentro de GlitchTip, permitiendo capturar y centralizar las excepciones de cada componente de forma aislada.

Este enfoque permite detectar errores en tiempo real, visualizar el stack trace completo de cada excepción y facilitar la trazabilidad de fallos en un sistema distribuido, sin depender de la revisión manual de logs en cada pod.

5. Evaluación Ética y Técnica

- **5.1 Privacidad y Seguridad de los Datos**

Desde una perspectiva ética, la protección de la información relacionada con ventas, inventario y usuarios es un aspecto crítico dentro del sistema. La arquitectura basada en microservicios permite segmentar los datos, reduciendo la exposición innecesaria de información sensible entre componentes.

- **5.2 Aislamiento de Responsabilidades**

La implementación de una arquitectura basada en microservicios con base de datos independiente por servicio permite que cada componente gestione su propia información de forma aislada. Esto reduce el impacto de posibles fallos o vulnerabilidades, evitando que una brecha de seguridad afecta a todo el sistema.

- **5.3 Control de Acceso**

El sistema contempla mecanismos de autenticación centralizados a través del Backend For Frontend (BFF), el cual actúa como punto de entrada controlado hacia los microservicios. Esto permite restringir el acceso a las funcionalidades del sistema según el tipo de usuario, reduciendo la exposición directa de los servicios internos.

6. Descripción de componentes:

La arquitectura está compuesta por diferentes microservicios independientes, donde cada componente tiene una responsabilidad específica dentro del sistema, estos son:

Frontend: Es la interfaz de usuario que se encarga de permitir la interacción con la plataforma. Consume los servicios expuestos por el Backend For Frontend (BFF) para obtener información y ejecutar operaciones del sistema.

Backend For Frontend (BFF): Actúa como punto único de entrada entre el frontend y los microservicios internos. Su función es recibir las solicitudes del usuario, validarlas y comunicarse con los servicios correspondientes mediante llamadas internas. Además, centraliza el manejo de errores y simplifica la comunicación con el frontend.

GlitchTip: Servicio de infraestructura encargado del monitoreo y registro centralizado de errores del sistema. Recibe reportes de excepciones desde cada microservicio y el BFF, permitiendo identificar, clasificar y dar seguimiento a los fallos ocurridos en producción de forma centralizada, sin la necesidad de acceder individualmente a los logs de cada componente.

ms-login: Microservicio encargado de la gestión de usuarios y autenticación del sistema. Permite registrar usuarios, realizar inicio de sesión y generar tokens JWT para validar el acceso a los distintos recursos según los permisos correspondientes.

ms-inventory: Microservicio responsable de la administración del inventario. Permite crear, consultar, actualizar y eliminar productos, manteniendo la información asociada al stock disponible.

ms-cart: Microservicio encargado de gestionar el carrito de compras del usuario. Permite agregar productos, modificar cantidades y eliminar elementos antes de confirmar una compra.

ms-order: Microservicio encargado del registro y administración de órdenes de compra. Almacena la información relacionada con las compras realizadas por los usuarios.

ms-reporting: Microservicio encargado de la generación y gestión de reportes del sistema. Permite obtener información procesada y entregar resultados en formatos visuales o descargables para facilitar el análisis de los datos.

ms-analytics: Microservicio responsable del procesamiento y análisis de la información del sistema. Se encarga de calcular indicadores clave (KPI), generar métricas y entregar información relevante para apoyar la toma de decisiones.

ms-data-aggregation: Microservicio encargado de recopilar y centralizar información proveniente de los distintos servicios del sistema. Permite consolidar los datos de inventario, ventas y operaciones para facilitar su consulta y procesamiento.

Base de datos por microservicio: Cada microservicio posee su propia base de datos independiente utilizando MongoDB. Esto permite mantener el aislamiento de datos, mejorar la escalabilidad y evitar dependencias directas entre servicios.

7. Conclusión:

El diseño propuesto para la plataforma del Grupo Cordillera permite abordar de manera eficiente la integración y gestión de información proveniente de distintos sistemas de la organización. La arquitectura basada en microservicios facilita la escalabilidad, el mantenimiento independiente de cada componente y la incorporación de nuevas funcionalidades sin afectar al sistema completo.

La implementación del patrón Backend For Frontend (BFF) mejora la comunicación entre el frontend y los microservicios, centralizando las solicitudes y simplificando el consumo de los servicios. Asimismo, el uso del patrón Repository contribuye a una correcta separación de responsabilidades en el acceso a datos.

En resumen, la solución propuesta permite mejorar la disponibilidad de la información, optimizar la toma de decisiones basada en datos y asegurar una estructura tecnológica más ordenada, flexible y preparada para futuras ampliaciones.